How prompts are assigned trait-axis coordinates

`infl_ens` methodology note

August 14, 2026

Abstract

This note records the *current* pipeline that maps a text prompt to a point $b^* \in [0,1]^L$ in the learned safety trait space. The seven-axis setup used by `seven_axis_pair_merge_split` has $L = 7$. Axes are external, benchmark-supervised summaries—not claimed transformer-internal features. Routing then evaluates influence shares $G_i(x_i, b^*)$ at that continuous coordinate.

1 Overview

The assignment pipeline is:

$$\text{prompt } q \xrightarrow{E} e_q \xrightarrow{\text{learned axes}} c^{\text{base}} \xrightarrow{\text{residualize}} c \xrightarrow{\text{stretch}} b^* \in [0,1]^L.$$

Implementation entry points: `build_safety_trait_space_bundle` (`src/infl_ens/data/benchmarks/safety_tr` and the cached wrapper `build_or_load_safety_trait_space`.

Two distinct objects live in the same box $[0,1]^L$:

- **Prompt coordinate** $b^* = \text{project}(q)$ — where this query sits.
- **Agent position** x_i — where specialist i chooses to compete.

Only b^* is produced by the methodology below. Agent positions are set by theory init / closed-loop updates and are *not* label assignments.

2 Datasets and axis labels

Axes are learned in config order. Each axis has a labelled benchmark split.

Axis label	Dataset	Positive class (high end of axis)
harm	BeaverTails	unsafe / not-safe responses
hallucination	HaluEval (QA, dialogue)	incorrect responses
jailbreak	JBB-Behaviors	harmful goals (vs benign)
privacy	AI4Privacy	high PII character density
overrefusal	OR-Bench	seemingly-toxic but benign
injection	Prompt Injection	injection prompts
policy	Do-Not-Answer	policy-violating prompts

For Fisher fitting, `harm` and `hallucination` currently use *response* embeddings as the supervised target; the other five axes use *prompt* embeddings. At routing time, **every** query is projected from its **prompt** text.

3 Embedding

Let E be a frozen sentence encoder (`sentence-transformers/all-MiniLM-L6-v2` in the seven-axis config). A string t maps to

$$e = E(t) \in \mathbb{R}^D.$$

There is no L2-normalization in the learned-axis path. Unique strings are encoded once and cached.

4 Learning one axis direction

For axis ℓ, split the labelled embeddings into positives and negatives by a score threshold τ (config: $\tau = 0.5$):

$$\mathcal{P}_\ell = \{e : s(e) \geq \tau\}, \qquad \mathcal{N}_\ell = \{e : s(e) < \tau\}.$$

For discrete labels, $s \in \{0, 1\}$. For AI4Privacy density mode,

$$s(x) = \min\Big(1, \frac{\sum_s (\text{end}_s - \text{start}_s)}{\max(1, |x|)}\Big).$$

Compute class means μ_+, μ_- and a pooled covariance S_w. Apply Ledoit–Wolf-style shrinkage (code default $\alpha = 0.1$):

$$S_{\text{reg}} = (1 - \alpha) S_w + \alpha \, \tfrac{\text{tr}(S_w)}{D} \, I, \qquad \tilde{w} = S_{\text{reg}}^{-1}(\mu_+ - \mu_-).$$

Then Gram–Schmidt-orthogonalize $\tilde{w}$ against previously learned directions and unit-normalize to obtain w_ℓ.

5 Calibration to $[0, 1]$

Project the negative and positive sets onto w_ℓ. Set

$$\ell_\ell = \text{percentile}_q(w_\ell^\top \mathcal{N}_\ell), \qquad h_\ell = \text{percentile}_{100-q}(w_\ell^\top \mathcal{P}_\ell),$$

with $q = 5$ (code default). The base coordinate is

$$c_\ell^{\text{base}} = \text{clip}_{[0,1]}\Big(\frac{w_\ell^\top e - \ell_\ell}{h_\ell - \ell_\ell}\Big).$$

6 Mode alignment (optional)

To mix *label* separation with *benchmark-of-origin* separation, form a second Fisher direction w^{mode} contrasting this split's prompts against all other splits' prompts, then

$$w_\ell \leftarrow (1 - \beta)\, w_\ell^{\text{label}} + \beta \, w_\ell^{\text{mode}},$$

re-orthogonalize, and recalibrate. In the seven-axis config, $\beta = 0.35$ by default and $\beta = 1.0$ for jailbreak, privacy, overrefusal, injection, and policy (pure mode directions for those five).

7 Coordinate residualization (optional)

With `coordinate_residualize: true`, later axes are decorrelated from earlier ones. On the calibration corpus, for $j > 0$, fit

$$\hat{c}_j = a_j + c_{0:j}^\top \beta_j$$

by least squares on base scores, set residual $r_j = c_j^{\text{base}} - \hat{c}_j$, and recalibrate r_j with that axis's labels to obtain the final pre-stretch coordinate $c_j \in [0, 1]$. Order follows the benchmark list (harm first, policy last).

8 Stretch

A concave stretch pushes moderate values toward one:

$$c^{(\gamma)} = 1 - (1 - c)^\gamma, \qquad \gamma > 0.$$

Seven-axis defaults: $\gamma = 1.5$ for harm; $\gamma = 2.5$ for the other six axes. The runtime projector returns the stretched vector b^*.

9 Runtime projection of a new prompt

Given query q:

1. Embed $e_q = E(q)$ (prompt text only).
2. For each axis: base score $w_\ell^\top e_q$, then residualize if enabled.
3. Apply per-axis stretch; clip to $[0, 1]^L$.

This is `TraitSpace.project`. It does *not* snap to the KDE grid.

10 Resource density $B(b)$ (not per-prompt assignment)

Separately, calibration coordinates are smoothed onto a Cartesian grid to form the resource measure B used for expected utilities:

$$B(b_k) \propto \sum_n \exp\Big(-\tfrac{1}{2}\frac{\|b_k - c_n\|^2}{h^2}\Big), \qquad \sum_k B(b_k) = 1.$$

Seven-axis config uses $n_{\text{grid}} = 3$ (so $K = 3^7 = 2187$ cells) and $h = 0.08$. These knobs affect landscape integrals $u_i = \sum_k B(b_k) G_i(x, b_k)$, **not** the continuous coordinate b^* of an individual prompt.

11 How routing uses the coordinates

With Gaussian influence,

$$\log f_i(x_i, b) = -\tfrac{1}{2}(x_i - b)^\top \Sigma^{-1}(x_i - b), \qquad G_i(\mathbf{x}, b) = \frac{f_i(x_i, b)}{\sum_j f_j(x_j, b)}.$$

Live routing: compute $b^* = \text{project}(q)$, then allocate with $G_i(\mathbf{x}, b^*)$ (proportional policy samples $\propto G_i$). Specialists receive prompts near their positions x_i; that is the mechanism by which axis geometry becomes a training curriculum.

12 Caveats

- Axes are external proxies (labels + MiniLM); they can confound topic, style, and difficulty.
- Harm / hallucination directions are fit on responses but applied to prompts.
- Mode weight 1.0 on five axes makes those coordinates largely "which benchmark did this come from?" separators.
- Residualization is order-dependent; Gram–Schmidt on directions does not guarantee independent calibrated scores without residualize.
- A coarse $n_{\text{grid}} = 3$ resource is a poor visualization of density; routing itself still uses continuous b^*.

Source files

`safety_trait_space.py`, `trait_space.py`, `trait_space_cache.py`, `configs/benchmark/router/seven_axis_p`